

Billing Email Automation

Feature built for Hope Hospital - Task Optimizer Module

Date: 06 June 2026

What We Built

A complete email-handling workflow inside the Adamrit HMS that connects directly to the info@hopehospital.com Gmail inbox. Staff can fetch incoming corporate, TPA and billing emails with one click, review each email, edit an auto-generated draft reply, re-phrase it using AI, and approve or reject - all from inside the application. No separate email server is needed and the core features run completely free.

How To Use It

1. Go to Task Optimizer -> Billing -> Corporate Emails
2. Click "Start Checking Mail" - up to 50 emails are fetched from info@hopehospital.com
3. Click any email card to open it - two tabs appear: Full Email and Draft Reply
4. Use the category chips (TPA / Corporate / Billing / Urgent / General) to filter
5. In the Draft Reply tab, edit the reply or click "Re-phrase" to change its style
6. Click "Approve & Send" to open Gmail pre-filled, or "Reject" to dismiss

Key Features

1. One-Click Mail Fetch

- Connects silently to Gmail using a stored OAuth2 refresh token - no login popup each time
- Fetches the latest 50 emails and reads the full body of each
- Read-only access - no email is ever deleted or modified in Gmail

2. Smart Classification (Free)

- Each email auto-tagged by keyword: TPA, Corporate, Billing, Urgent, or General
- Urgency level detected: High / Medium / Low

3. Auto Draft Replies

- A professional reply is pre-written for every email based on its category
- The draft is also created directly inside the Gmail Drafts folder, properly threaded

4. AI Re-phrase (Gemini)

- Re-phrase button rewrites the reply in 4 styles: Standard, Formal, Brief, Friendly
- Powered by Google Gemini - the same key already used for discharge summaries

5. Human Approval Workflow

- Nothing is sent automatically - staff must review and approve every reply
- Approve & Send opens Gmail compose pre-filled; staff clicks Send
- Reject marks the email as handled without replying

6. Permanent De-duplication

- Each Gmail message has a unique ID stored on first fetch
- Clicking "Start Checking Mail" repeatedly never creates duplicate entries

Technical Architecture

Browser

```
|
+- OAuth2 token exchange (silent, no popup)
|   refresh_token -> Google -> access_token (55-min cache)
|
+- Gmail API (read-only fetch + draft creation)
|   GET  /users/me/messages?maxResults=50
|   POST /users/me/drafts  (threaded reply)
|
+- Keyword classifier -> draft reply template
|
+- Gemini AI (re-phrase in 4 styles)
|
+- Supabase (admin client, bypasses RLS)
    email_inbox table -> stores email + draft
```

Files Created / Changed

BillingPanel.tsx	Main UI - list, filters, card expand, approve/reject/re-phrase
useGmailChecker.ts	Gmail fetch, dedup, draft creation, Gemini re-phrase
adminClient.ts	Supabase service-role client (bypasses RLS)
setup-oauth.js	One-time Gmail OAuth token setup (auto-opens browser)
email_inbox migrations	Database table + permissions
billing-email-automation.md	Full feature documentation

Database Table: email_inbox

<code>from_email / from_name</code>	Sender details
<code>subject / body_preview</code>	Email subject and full body
<code>category / urgency</code>	Auto-classified tags
<code>draft_reply</code>	AI / template reply - editable
<code>status</code>	pending / approved / rejected / sent
<code>approved_by</code>	Stores Gmail msg ID for dedup; "staff" on approval

Setup Notes

Environment Variables (in .env)

```
VITE_GMAIL_CLIENT_ID=...
VITE_GMAIL_CLIENT_SECRET=...
VITE_GMAIL_REFRESH_TOKEN=...
VITE_GMAIL_USER_EMAIL=info@hopehospital.com
VITE_SUPABASE_SERVICE_ROLE_KEY=...
VITE_GEMINI_API_KEY=...
```

If the Gmail token expires

Run: `node scripts/email-automation/setup-oauth.js`

A browser opens automatically -> log in as `info@hopehospital.com` -> click Allow. The new token saves to `.env` automatically.

Status

- All code committed and merged into the main branch
- Pushed to GitHub - Vercel will auto-deploy
- `email_inbox` table created in Supabase (project `xvkxccqaopbnkvwgyfjv`)
- Feature is live and working on `https://localhost:8080`